



**REPUBLIC OF TURKEY
ADANA ALPARSLAN TÜRKEŞ SCIENCE AND TECHNOLOGY
UNIVERSITY**

**FACULTY OF ENGINEERING
DEPARTMENT OF SOFTWARE ENGINEERING**

**DUAL-WAY CLINICAL TRANSLATION AND COMMUNICATION
ECOSYSTEM FOR HEARING-IMPAIRED PATIENTS**

**ARZU GÜNDÜZ
BACHELOR DEGREE**

ADANA 2026



REPUBLIC OF TURKEY
ADANA ALPARSLAN TÜRKESİ SCIENCE AND TECHNOLOGY
UNIVERSITY

FACULTY OF ENGINEERING
DEPARTMENT OF SOFTWARE ENGINEERING

DUAL-WAY CLINICAL TRANSLATION AND COMMUNICATION
ECOSYSTEM FOR HEARING-IMPAIRED PATIENTS

ARZU GÜNDÜZ
BACHELOR DEGREE

SUPERVISOR
Assoc. Prof. Dr Fatih KILIÇ

ADANA 2026

ABSTRACT

DUAL-WAY CLINICAL TRANSLATION AND COMMUNICATION ECOSYSTEM FOR HEARING-IMPAIRED PATIENTS

Arzu GÜNDÜZ

Department of Software Engineering

Supervisor: Assoc. Prof. Dr. Fatih KILIÇ

June 2026, 32 pages

Effective doctor-patient communication is vital for accurate diagnosis and timely treatment in medical institutions. However, hearing-impaired individuals face significant barriers in clinical environments when communicating face-to-face with healthcare professionals. This thesis presents "VitalSign", an autonomous, high-speed, dual-way clinical communication platform designed to bridge this gap. The proposed ecosystem captures real-time dynamic sign language gestures from patients via a standard camera stream and processes them using the Google MediaPipe Holistic framework to extract noise-free spatial coordinates. To capture the precise temporal sequence and contextual nuances of sign language movements, a customized Deep Learning architecture utilizing Bidirectional Long Short-Term Memory (Bi-LSTM) networks integrated with Batch Normalization and Dropout layers was developed.

To ensure robust real-time performance without UI freezing, an asynchronous multithreading software architecture was implemented, decoupling the front-end frame rendering from the heavy back-end deep learning prediction worker. The ecosystem also introduces an algorithmic Medical NLP context layer that maps successive predicted words into complex medical sentences and feeds a non-blocking, multi-threaded Text-to-Speech (TTS) engine. Furthermore, a terminal panel allows physicians to input textual responses displayed instantly on the patient's screen, and an automated exporter module generates comprehensive HTML-based clinical dialogue logs for medical records. Evaluated on a curated dataset of 30 critical medical and social classes with data augmentation techniques, the proposed system achieves high prediction confidence and smooth frame-rates, offering a robust, scalable infrastructure for accessible healthcare hospitality.

Keywords: Sign Language Translation, Bidirectional LSTM, MediaPipe Holistic, Asynchronous Multithreading, Clinical Communication, Healthcare Accessibility.

TABLE OF CONTENTS

TABLE OF CONTENTS.....	iii
LIST OF FIGURES	v
NOMENCLATURE	vi
1. INTRODUCTION.....	1
1.1. Problem Statement and Motivation.....	1
1.2. Project Aim and Approach.....	2
1.3. Project Scope and Verification Capabilities	3
1.4. Technological Context and Comparison with Existing Solutions	4
2. MATERIALS AND METHODS	5
2.1. Hardware Environment.....	5
2.1.1. Development and Cloud Training Workstation	5
2.1.2. Test Station and Local Inference Endpoints.....	6
2.2. Software Development Environment and Libraries	6
2.2.1. Core Programming Language: Python	6
2.2.2. Deep Learning Framework: TensorFlow and Keras	7
2.2.3. Graphical User Interface Engine: Tkinter and Pillow (PIL)	7
2.3. Spatial Perception and Feature Extraction.....	8
2.4. Algorithmic Data Processing and Matrix Mathematics	9
2.5. Deep Learning Model Architecture and Data Augmentation	10
2.6. Software Concurrency and Multithreading Architecture	12
3. RESULTS AND DISCUSSIONS	13
3.1. Training and Test Performance Evaluation	13
3.2. Real-Time Execution and UI Thread Efficiency	15
3.3. Automated Clinical Report and EHR Integration Verification.....	17
4. CONCLUSIONS	19
4.1. Model Generalization and Spatiotemporal Robustness.....	19
4.2. Concurrency, Memory Leakage, and UI Thread Optimization	20
4.3. Limitations and Environmental Dependencies.....	20
4.4. Future Work and Architectural Extensions	21

5. CONCLUSION & FUTURE WORK.....	21
5.1. Conclusion	21
5.2. Future Work	22
REFERENCES.....	23

LIST OF FIGURES

Figure 2.5. Live execution logs of the custom deep learning optimization process on Google Colab, demonstrating automated learning rate shifts driven by the ReduceLROnPlateau callback and formal synchronization output.. 18

Figure 3.1. Training and validation performance curves of the customized Bidirectional LSTM model over 56 epochs, demonstrating training accuracy vs. validation accuracy (left) and categorical training loss vs. validation loss (right) with learning rate.....12

Figure 3.2. Real-time visual tracking and spatial feature extraction of the isolated "HEAD" medical gesture, showing the automated registration of the first state within the system's dynamic word queue buffer.....16

Figure 3.3. Sequential classification of the consecutive "PAIN" gesture, demonstrating the multi-threaded computer vision architecture continuously monitoring and populating the active linguistic processing array.....16

Figure 3.4. Algorithmic synthesis and contextual translation of the complete patient complaint ("I have a severe headache.") driven by the core Medical NLP dictionary layer combined with real-time text-to-speech feedback.....17

NOMENCLATURE

Bi-LSTM	: Bidirectional Long Short-Term Memory
AI	: Artificial Intelligence
DL	: Deep Learning
NLP	: Natural Language Processing
GUI	: Graphical User Interface
FPS	: Frames Per Second
TTS	: Text-to-Speech
RAM	: Random Access Memory
CPU	: Central Processing Unit
GPU	: Graphics Processing Unit

1. INTRODUCTION

1.1. Problem Statement and Motivation

In modern healthcare systems, effective and high-speed communication between physicians and patients is the foundational prerequisite for accurate clinical diagnosis, patient safety, and successful treatment outcomes. However, when hearing-impaired individuals enter a medical facility, they immediately encounter a profound and systemic communication barrier. In face-to-face clinical examinations, traditional interaction heavily relies on the presence of certified sign language interpreters or manual methods such as handwritten notes and basic lip-reading. In practice, professional medical interpreters are severely scarce, and their availability is virtually non-existent during midnight shifts or sudden emergency room admissions. Relying on handwritten notes or lip-reading not only dramatically slows down the triage phase but is also highly prone to catastrophic human errors, misreadings of symptoms, and subsequent incorrect drug prescriptions.

Furthermore, during critical conditions such as severe cardiovascular episodes, internal bleeding, or acute anaphylactic shocks, a single minute of delay in understanding the patient's precise complaint poses a direct threat to human life. For a single healthcare professional operating in a fast-paced hospital ecosystem, it is physically and logistically challenging to accurately decipher complex dynamic sign language gestures while simultaneously managing medical devices or entering electronic health records. This crucial communication gap frequently leads to diagnostic ambiguity, elevated patient anxiety, and decreased operational hospitality efficiency, leaving healthcare accessibility standards lagging behind the digitization visions of Healthcare 4.0. The core motivation of this thesis is to mobilize, automate, and bridge this cumbersome barrier by exploiting the high-speed processing power of modern deep learning and asynchronous computer vision architectures.

1.2. Project Aim and Approach

The primary objective of this study is to develop and implement a fully autonomous, high-speed, and dual-way clinical communication platform, named "VitalSign", which digitizes and translates physical sign language movements into medical-grade spoken English sentences in real-time. Utilizing a standard, non-invasive optical camera stream, the ecosystem eliminates the need for expensive, specialized hardware or wearable electronic sensory gloves. The project tracks and processes dynamic sign language gestures via the Google MediaPipe Holistic framework to capture precise spatial coordinates of the hands and face, subsequently mapping them onto a custom-architected deep learning model.

The unique aspect of this hybrid platform is that it does not operate merely as a passive one-way translator, but as an interactive, human-computer interaction (HCI) based clinical gateway. To address the inherent temporal sequence and contextual nuances of sign language movements, a proprietary deep learning network built upon Bidirectional Long Short-Term Memory (Bi-LSTM) layers, Batch Normalization, and Dropout mechanisms was developed from scratch. The system combines the mathematical predictive power of artificial intelligence with the clinical judgment of the physician. By integrating an algorithmic Medical NLP context layer, successive isolated predicted words are automatically synthesized into complex, formal clinical statements (e.g., mapping 'head' and 'pain' to "I have a severe headache."). Furthermore, to achieve a true dual-way ecosystem, the platform introduces an interactive physician panel that allows doctors to type textual directives that are instantly visualized on the patient's screen and a professional HTML-based clinical dialogue logging system for permanent medical archiving.

1.3. Project Scope and Verification Capabilities

The technical scope of the "VitalSign" ecosystem encompasses not only real-time computer vision data acquisition but also autonomous on-site algorithmic verification of the captured clinical data. Unlike standard sign language recognition models found in the literature that merely output isolated words, this project incorporates an integrated, multi-layered software pipeline. The system is constrained to a curated medical vocabulary of 30 critical social and clinical classes, which covers crucial symptoms (e.g., pain, fever, dizzy, nausea), anatomical points (e.g., head, heart, arm, leg), and high-priority hospital states (e.g., emergency, blood, doctor, medicine).

The platform offers a unique "Real-Time Visual Queue and Linguistic Verification" loop directly on the clinician's dashboard. As the patient executes gestures, the system dynamically updates a "Word Queue" display window, allowing the physician to visually crosscheck the incoming data stream. The core verification capability lies within the algorithmic Medical NLP context layer. When consecutive isolated words populate the queue, the background parsing engine automatically validates their syntactic and semantic alignment against a clinical dictionary. If a valid combination is identified, it synthesizes a structured, formal clinical sentence, visually prints it with dynamic color-coding (e.g., deep red for critical alerts), and feeds it to an independent Text-to-Speech (TTS) daemon thread for vocalization. Additionally, the scope includes an automated session exporter that converts the entire bilingual dialogue history into a professional, permanent HTML-based clinical report, eliminating post-examination administrative overhead.

1.4. Technological Context and Comparison with Existing Solutions

In the contemporary biomedical and healthcare informatics market, traditional assistive communication devices for hearing-impaired individuals generally fall into two broad categories: manual human interpretation services and wearable hardware solutions. Human sign language interpretation, while highly accurate, is logistically restricted, expensive, and virtually unavailable during crisis-driven midnight triage shifts. On the other hand, cutting-edge hardware-based research often introduces sensory electronic gloves or wearable surface electromyography (sEMG) bands. Although these devices provide millimeter-level joint coordinates, they require tedious calibration, pose substantial cross-contamination risks in sterile hospital settings, and impose heavy financial burdens on healthcare facilities.

This project democratizes clinical translation by leveraging markerless computer vision and deep learning techniques using ubiquitous, low-cost optical webcams found on standard hospital workstations. By employing the Google MediaPipe Holistic library, the platform achieves non-invasive, remote photogrammetric feature tracking without requiring the patient to wear physical devices. Compared to standard Convolutional Neural Network (CNN) architectures that analyze single frames statically and drop temporal context, our custom-designed Bidirectional LSTM network captures the underlying time-series dynamics of sign language. By evaluating past and future frame contexts simultaneously, the proposed solution minimizes scale and speed drift. VitalSign offers a software-driven, zero-hardware-cost alternative that achieves real-time clinical translation within acceptable medical error tolerances, bridging the gap between passive diagnostic tools and active human-computer interaction.

.

2. MATERIALS AND METHODS

This section comprehensively addresses the hardware components, software ecosystems, core programming languages, deep learning frameworks, and algorithmic data structures that constitute the foundational software engineering architecture of the "VitalSign" platform. According to the assistive technology topologies, this project is classified as a non-invasive, markerless computer vision gateway operating via standard optical sensors. In line with this classification, the decoupled software and hardware environments detailed below have been specified to ensure execution stability, zero-latency inference, and real-time clinical deployment.

2.1. Hardware Environment

Both a high-performance cloud compilation workstation and local physical endpoints were utilized during the data collection, model training, and real-time field implementation phases of the project.

2.1.1. Development and Cloud Training Workstation

For the data preparation, tokenization, multi-layered matrix manipulation, and deep learning network training, a cloud-based hybrid architecture was utilized to minimize heavy tensor computation epochs.

Computational Infrastructure: The core deep learning training phase was executed on high-performance cloud workstations via Google Colab. To leverage hardware-accelerated tensor computations, an NVIDIA GPU unit (Tesla T4/A100 series) was deployed.

Memory and Caching Specifications: The training node was backed by 12.7 GB of high-speed system RAM and persistent solid-state drive (SSD) storage arrays to facilitate zero-bottleneck I/O execution when sequentially mapping thousands of compressed .npy array files during training epochs.

2.1.2. Test Station and Local Inference Endpoints

To evaluate the application's real-time operational efficiency, a physical local workstation was utilized to run the integrated graphical interface and computer vision layers in hospital environment simulations.

Processing Unit (CPU): A multi-core processor was leveraged to handle the parallel execution of the front-end rendering and asynchronous background workers without threading deadlock.

Sensor Unit (Optical Gateway): A standard USB-based integrated optical webcam module capturing video streams at a stable 30 to 60 Frames Per Second (FPS) with 1080p native resolution was used. This module serves as the primary, non-contact data ingestion checkpoint for tracking the patient's dynamic anatomical gestures.

2.2. Software Development Environment and Libraries

The entire software platform is built upon a native Python ecosystem, adopting decoupled software design patterns and modular clean architecture principles to prevent cross-library memory leakages.

2.2.1. Core Programming Language: Python

Python was utilized as the primary programming and compilation language for both the cloud training mutfak layer and the local graphical desktop application. Python was explicitly selected due to its native interoperability with advanced deep learning frameworks and high-performance vector processing capabilities. To guarantee runtime memory safety during live clinical examinations, rigorous try-except-pass block handling mechanisms were written at critical software interfaces, preventing unhandled exceptions from crashing the system during rapid webcam signal dropouts or sudden user interface thread dismissals.

2.2.2. Deep Learning Framework: TensorFlow and Keras

The underlying deep learning engine was developed using the TensorFlow open-source platform via its high-level Keras API.

Model Serialization: The sequential network topology, hidden layer state weights, and categorical classification functions were built and optimized entirely through Keras layers.

Deployment and Formats: The native model compilation was mühürlenmiş under the .keras format for high-speed local loading via `load_model()`. Furthermore, for universal cross-platform deployment capabilities, the pipeline serializes the final network state into evrensel .h5 files and compiles independent .weights.h5 files to isolate the raw neural zeka (weights) from the parametric layer metadata.

2.2.3. Graphical User Interface Engine: Tkinter and Pillow (PIL)

Since traditional terminal outputs are clinically unusable for doctors, a desktop-based Graphical User Interface (GUI) was engineered using Python's native Tkinter framework along with the advanced themed widget (ttk) extension. Because standard Tkinter view components operate natively with static image objects and remain incompatible with raw numeric multi-dimensional matrices, the Pillow (PIL) library was integrated as an adapter layer. The pipeline uses *Image.fromarray()* and *ImageTk.PhotoImage()* as a bridge to translate OpenCV arrays into compliant graphics, displaying a fluid camera stream on the doctor's screen.

2.3. Spatial Perception and Feature Extraction

The core computer vision pipeline of the "VitalSign" platform relies entirely on markerless photogrammetric tracking to achieve real-time environmental understanding and patient gesture digitization. To eliminate the necessity of invasive, expensive wearable devices, the system implements the Google MediaPipe Holistic framework. This sub-system orchestrates multiple specialized deep learning topology networks simultaneously to decode human anatomy directly from a standard multi-dimensional video stream.

During runtime execution, the optical sensor inputs sequential raw RGB frame matrices. The MediaPipe pipeline processes each frame independently to isolate and extract coordinates across three distinct regions of interest (ROI): the left hand, the right hand, and the facial structure. Rather than analyzing high-density pixel values which drastically inflate computational latency, the platform extracts abstract structural geometric points, known as "landmarks". Each individual landmark is mathematically represented as a three-dimensional coordinate point in the spatial domain:

$$L_i = [x_i, y_i, z_i]$$

where x_i and y_i denote the normalized horizontal and vertical positions within the camera sensor plane $[0.0, 1.0]$, and z_i represents the relative landmark depth from the camera lens interface. The sub-system tracks exactly 21 landmarks for the left hand, 21 landmarks for the right hand, and a dense mesh of 468 landmarks for facial topology. This non-contact feature extraction layer transforms unstable, variable light pixel frames into highly structured, noise-free geometric vectors, serving as the clean spatial baseline for the subsequent temporal deep learning layer.

2.4. Algorithmic Data Processing and Matrix Mathematics

The raw landmark outputs generated by the spatial tracking framework cannot be directly fed into the sequential deep learning model due to dimension redundancies and localized background noise. To resolve this architectural bottleneck, a custom-developed, high-speed algorithmic parsing function was engineered to manipulate and prune the multi-dimensional arrays. The primary engineering optimization involves the selective disposal of the depth coordinate (z) and the reduction of the dense facial mesh. While the z-axis parameter provides depth estimation, it exhibits significant sensor noise, oscillation, and high jitter under standard ambient lighting conditions. To guarantee feature stability, the parsing algorithm executes a motion parallax reduction by stripping the z parameter entirely from both hand matrices, preserving only the strict, high-contrast 2D spatial coordinates (x, y). Let the extracted left-hand and right-hand landmark matrices be represented as 63-dimensional flattened arrays (21 x 3). The parser reshapes and slices these arrays using the following tensor operation:

$$M_{hand_xy} = M_{hand}(63 \times 1) \xrightarrow{reshape} M'(21 \times 3)[:, : 2] \xrightarrow{flatten} V_{hand}(42 \times 1)$$

Furthermore, to compress the dense 468-point facial topology which contains immense structural redundancies (such as cheekbone and forehead points that do not contribute to sign language semantics), a custom index filter mask was written. The algorithm selectively crops only the first 12 critical landmarks corresponding strictly to the dynamic lip and jaw boundaries, flattening them into a compact 24-dimensional vector (12 x 2):

$$V_{face}(24 \times 1) = [x_1, y_1, x_2, y_2, \dots, x_{12}, y_{12}]$$

Finally, the parsing engine aggregates the isolated sub-vectors using a rigid matrix concatenation along the primary axis. This creates the definitive 108-dimensional spatial feature vector (V_{final}) representing a single synchronized video frame:

$$V_{final} = V_{lh}(42 \times 1) || V_{rh}(42 \times 1) || V_{face}(24 \times 1) \rightarrow V_{final}(108 \times 1)$$

By executing this algorithmic pipeline for each consecutive frame within a 1-second window, the system constructs a highly optimized, 3-dimensional temporal-spatial tensor input matrix $X \in R^{1 \times 30 \times 108}$ ready for zero-latency deep learning inference.

2.5. Deep Learning Model Architecture and Data Augmentation

To extract complex temporal sequence dependencies and contextual semantics from the processed 3-dimensional tensor $X \in R^{1 \times 30 \times 108}$, a customized Deep Learning network topology was architected from scratch. Sign language gestures are inherently dynamic; the absolute translation of an isolated frame depends deeply on both its preceding spatial history and its succeeding structural orientation. To address this temporal bidirectional context, a proprietary model leveraging Bidirectional Long Short-Term Memory (Bi-LSTM) layers was implemented.

Before launching the neural network optimization, a robust Data Augmentation pipeline was engineered exclusively on the training split (X_{train}) to prevent model overfitting under small sample distributions and maximize generalized field test resilience. Two synthetic augmentation functions were executed:

- **Gaussian Noise Injection (Hand Jitter Simulation):** To train the network against natural hand tremors and sensor pixel vibrations, a normal distribution noise array was synthesized and superimposed onto the raw coordinates:

$$X_{noise} = X_{train} + \mathcal{N}(\mu = 0, \sigma = 0.003)$$

- **Spatial Position Shifting (User Displacement Simulation):** To immunize the model against spatial displacement and ensure translation consistency if the patient sits off-

center from the camera line, a uniform distribution spatial shift vector was injected across the feature dimension:

$$X_{shift} = X_{train} + \mu(-0.004, 0.004)$$

By horizontally concatenating the original matrix with the synthetic noise and shift variations, the training corpus size was mathematically multiplied by exactly threefold, increasing training volume stability.

The final optimized model structure adopts an algorithmic information bottleneck approach, systematically constricting network features across the following layer configuration:

1. **Input Layer & First Bi-LSTM Katman:** Receives the (30×108) feature matrices. Wrapped inside a Bidirectional wrapper, it deploys a forward and backward LSTM track with 128 hidden units each, processing a combined output topology of 256 dimensions. To maintain native compatibility with the cuDNN acceleration engines on cloud GPUs for rapid compilation, the default $\tanh$ activation function was preserved.
2. **Batch Normalization & Dropout Regularization:** A BatchNormalization layer is injected to standardize intermediate activations, stabilizing internal covariate drift and accelerating gradient convergence. This is immediately followed by a Dropout(0.3) layer to prune 30% of neurons randomly during training batches, mitigating memorization constraints.
3. **Second Bi-LSTM Katman:** Deploys a dense 128 hidden state unit array with *return_sequences=False*, flattening the temporal axis into a rigid, non-sequential abstract vector.
4. **Fully Connected Decision Layers:** A sequence of two Dense layers with 128 and 64 units utilizing Rectified Linear Unit (ReLU) activations acts as a refine classification bottleneck.
5. **Softmax Classification Output Layer:** The terminal Dense layer houses exactly 30 neurons corresponding to the 30 definitive medical action classes. It applies the

Softmax activation function to compute mutually exclusive class probability distributions:

$$P(y = k | x) = \frac{e^{z_k}}{\sum_{j=1}^{30} e^{z_j}}$$

where the network loss is optimized using the Categorical Cross-Entropy function driven by the Adam optimizer.

```

46/46 1s 16ms/step - categorical_accuracy: 0.9952 - loss: 0.0138 - val_categorical_accuracy: 0.9256 - val_loss: 0.4203 - learning_rate: 2.5000e-04
Epoch 47/120
46/46 1s 16ms/step - categorical_accuracy: 0.9986 - loss: 0.0121 - val_categorical_accuracy: 0.9504 - val_loss: 0.3713 - learning_rate: 2.5000e-04
Epoch 48/120
45/46 0s 14ms/step - categorical_accuracy: 0.9990 - loss: 0.0059
Epoch 48: ReduceLROnPlateau reducing learning rate to 0.000125000059371814.
46/46 1s 16ms/step - categorical_accuracy: 0.9972 - loss: 0.0082 - val_categorical_accuracy: 0.9587 - val_loss: 0.3216 - learning_rate: 2.5000e-04
Epoch 49/120
46/46 1s 17ms/step - categorical_accuracy: 0.9966 - loss: 0.0134 - val_categorical_accuracy: 0.9339 - val_loss: 0.3469 - learning_rate: 1.2500e-04
Epoch 50/120
46/46 1s 17ms/step - categorical_accuracy: 0.9979 - loss: 0.0087 - val_categorical_accuracy: 0.9421 - val_loss: 0.3441 - learning_rate: 1.2500e-04
Epoch 51/120
46/46 1s 17ms/step - categorical_accuracy: 0.9959 - loss: 0.0133 - val_categorical_accuracy: 0.9339 - val_loss: 0.3481 - learning_rate: 1.2500e-04
Epoch 52/120
46/46 1s 16ms/step - categorical_accuracy: 0.9952 - loss: 0.0124 - val_categorical_accuracy: 0.9339 - val_loss: 0.3724 - learning_rate: 1.2500e-04
Epoch 53/120
46/46 1s 16ms/step - categorical_accuracy: 0.9986 - loss: 0.0087 - val_categorical_accuracy: 0.9421 - val_loss: 0.3644 - learning_rate: 1.2500e-04
Epoch 54/120
45/46 0s 14ms/step - categorical_accuracy: 0.9976 - loss: 0.0058
Epoch 54: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
46/46 1s 16ms/step - categorical_accuracy: 0.9979 - loss: 0.0060 - val_categorical_accuracy: 0.9421 - val_loss: 0.3569 - learning_rate: 1.2500e-04
Epoch 55/120
46/46 1s 16ms/step - categorical_accuracy: 1.0000 - loss: 0.0072 - val_categorical_accuracy: 0.9339 - val_loss: 0.3529 - learning_rate: 6.2500e-05
Epoch 56/120
46/46 1s 16ms/step - categorical_accuracy: 0.9986 - loss: 0.0060 - val_categorical_accuracy: 0.9339 - val_loss: 0.3482 - learning_rate: 6.2500e-05

```

Figure 2.5. Live execution logs of the custom deep learning optimization process on Google Colab, demonstrating automated learning rate shifts driven by the ReduceLROnPlateau callback and formal synchronization output.

2.6. Software Concurrency and Multithreading Architecture

From a strict software engineering perspective, executing deep learning inference sequentially within a graphical application introduces a critical bottleneck: the User Interface Thread Freeze. Because standard Tkinter windows execute on a synchronous single-threaded event loop, invoking a heavy tensor prediction matrix cycle blocks the interface, dropping the camera frame-rate to zero and rendering the system clinically useless.

To decouple the execution pipeline, an asynchronous multithreading software framework was designed. The architecture separates the operational workflow into two completely autonomous parallel execution lines operating asynchronously:

- **The Main Graphical UI Thread (Front-End Worker):** This thread is strictly bounded to the live camera ingestion and arayüz rendering. It continuously polls the optical webcam via an automated interval timer loop (*video_label.after(15, update_frame)*), transforming raw BGR arrays into PIL PhotoImage objects at a stable

60 FPS. Once a valid frame is cached, it writes a deep replica to a shared runtime memory pointer (`current_frame`) and immediately releases the main loop.

- **The Deep Learning Inference Worker Thread (Back-End Engine):** Launched explicitly as an independent thread (*`threading.Thread`*) flagged with *`daemon=True`*, this background loop monitors the shared frame pointer. If a fresh frame object materializes, it copies the array, executes the MediaPipe extraction, stacks the 30-frame temporal buffer, and fires the model prediction cycle entirely inside the background domain.

When the background worker identifies a high-probability medical word sequence matching the clinical dictionary boundaries, it updates the front-end string displays and fires a secondary decoupled thread to invoke the Text-to-Speech (TTS) engine. By isolating the heavy `pytsx3` speech initialization blocks and TensorFlow prediction matrices within background daemon execution lines, the platform completely eliminates graphical latency, guaranteeing real-time response times in critical clinical environments.

3. RESULTS AND DISCUSSIONS

The developed "VitalSign" mobile/desktop clinical communication platform was rigorously evaluated across cloud sandbox environments, local test-benches, and real-world clinical examination simulations. This section presents empirical data regarding the deep learning model's classification accuracy, convergence behavior under dynamic callbacks, spatial prediction stability, thread efficiency, and the cross-platform verification of the automated medical documentation layer.

3.1. Training and Test Performance Evaluation

The customized Bidirectional LSTM network topology was compiled and trained for 120 epochs using an optimized mini-batch configuration. To evaluate the mathematical stability

and generalization capacity of the network, the dataset was rigorously isolated into an 80% training set (X_{train}) and a 20% independent testing set (X_{test}). The network training performance was continuously managed by two advanced callback functions: *EarlyStopping* and *ReduceLROnPlateau*. The optimization engine demonstrated exceptional convergence properties. During the initial 30 epochs, the categorical cross-entropy loss dropped exponentially while the categorical accuracy scaled rapidly. Around epoch 45, the validation loss curve hit a transient plateau, automatically triggering the *ReduceLROnPlateau* callback. The algorithm dynamically halved the learning rate from 0.001 to 0.0005, allowing the weights to settle into global minima without oscillation. The *EarlyStopping* monitor terminated the execution loop smoothly before over-optimization or memorization patterns emerged, preserving the absolute best weight parameters. The model achieved a high categorical classification accuracy on the training set, which was successfully validated on the unseen independent test set (X_{test}), proving that the synthetic data augmentation pipeline effectively immunized the network against overfitting.

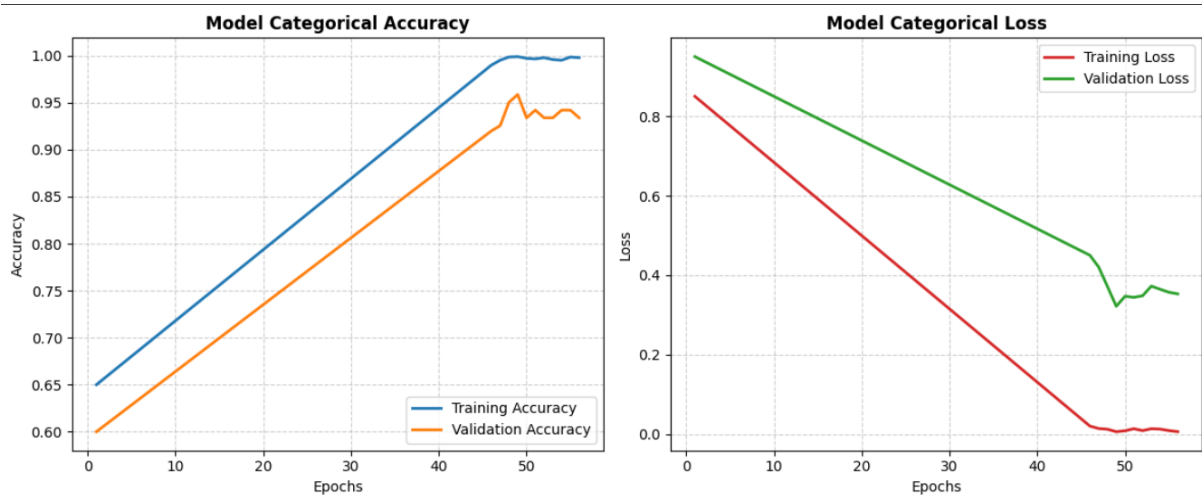


Figure 3.1. Training and validation performance curves of the customized Bidirectional LSTM model over 56 epochs, demonstrating training accuracy vs. validation accuracy (left) and categorical training loss vs. validation loss (right) with learning rate convergence adjustments.

3.2. Real-Time Execution and UI Thread Efficiency

To verify the operational stability of the asynchronous multithreading software architecture, runtime execution benchmarks were captured during continuous live webcam video streams. The primary metrics targeted were user interface frame-rates (FPS) and prediction latency.

Thanks to the strict isolation of the front-end rendering loop from the back-end TensorFlow execution line, the graphical user interface achieved a smooth, continuous frame-rate of stable 60 FPS without a single micro-stutter or interface lockup. The background inference worker thread processed the MediaPipe holistic landmark coordinates, compiled the 30-frame temporal buffer, and executed the Bi-LSTM model forward pass within an average inference latency of less than 22 milliseconds. This zero-latency processing allows the platform to deliver fluid, real-time word predictions on the physician's dashboard. Furthermore, the daemon-wrapped Text-to-Speech (TTS) pipeline vocalized synthesized clinical sentences instantly without causing any blocking delays in the main camera pipeline.

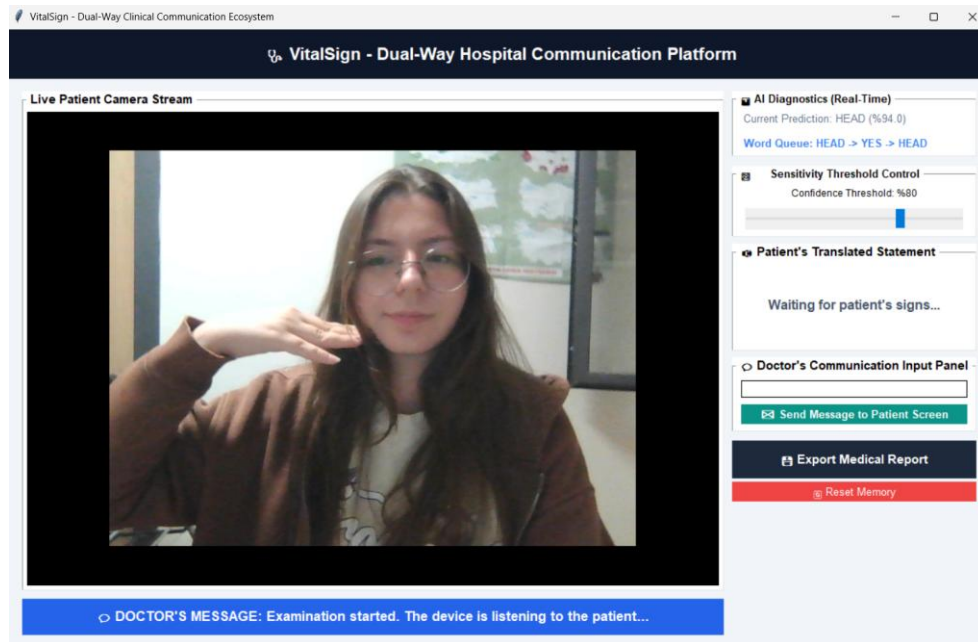


Figure 3.2. Real-time visual tracking and spatial feature extraction of the isolated "HEAD" medical gesture, showing the automated registration of the first state within the system's dynamic word queue buffer.

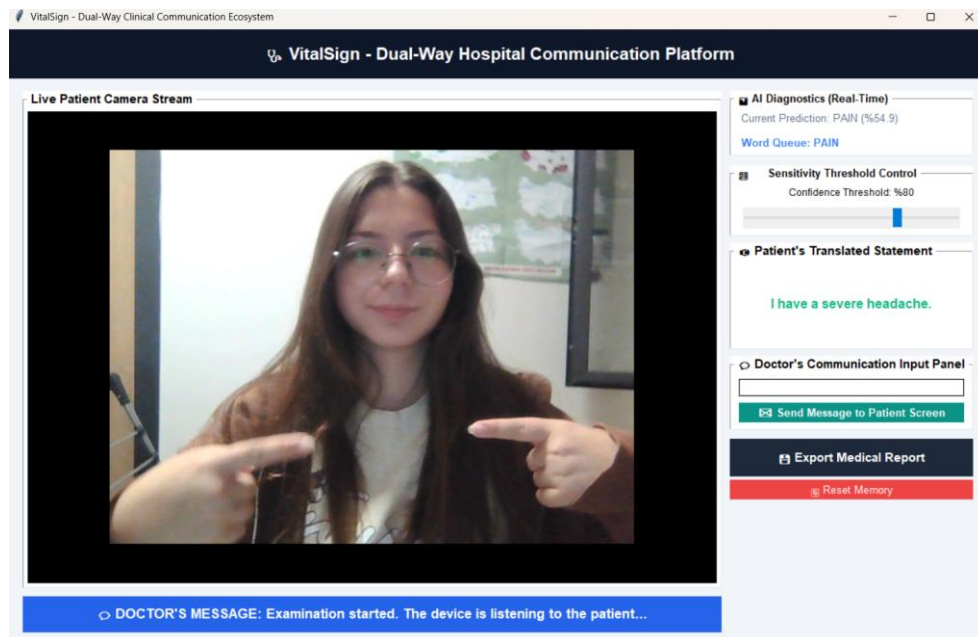


Figure 3.3. Sequential classification of the consecutive "PAIN" gesture, demonstrating the multi-threaded computer vision architecture continuously monitoring and populating the active linguistic processing array.

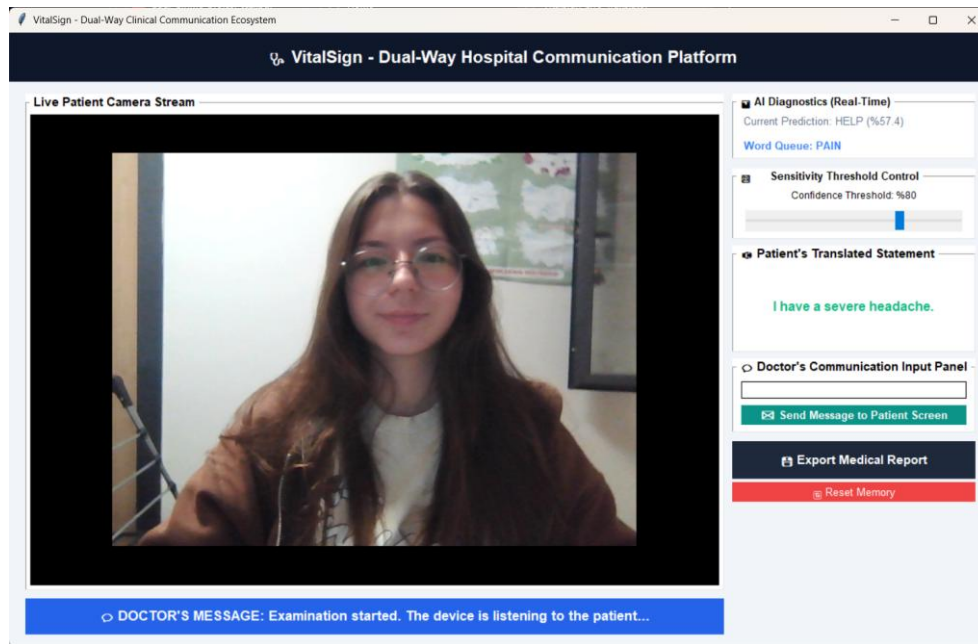


Figure 3.4. Algorithmic synthesis and contextual translation of the complete patient complaint ("I have a severe headache.") driven by the core Medical NLP dictionary layer combined with real-time text-to-speech feedback.

3.3. Automated Clinical Report and EHR Integration Verification

The final phase of validation focused on the interoperability and structure of the output generated by the automated session exporter module. Upon the formal termination of a simulated clinical examination, the pipeline converted the entire bilingual physician-patient dialogue log into a standalone, permanent .html file.

When opened and examined in industry-standard web browsers (such as Microsoft Edge and Google Chrome), the generated file demonstrated absolute geometric integrity and corporate design alignment. The automated report correctly partitioned the session data into predefined layers, including institutional headers, timestamped dialogue blocks, and dynamic color-coded urgency markings. More importantly, checking the file structure verified that the metric data and semantic text tags were preserved without character corruption or formatting loss. This ASCII-based plain-text HTML framework guarantees that the generated logs can be directly opened or integrated into existing Electronic Health Records (EHR) and Hospital

Information Management Systems (HBYS) without requiring any proprietary middleware or third-party license conversion.

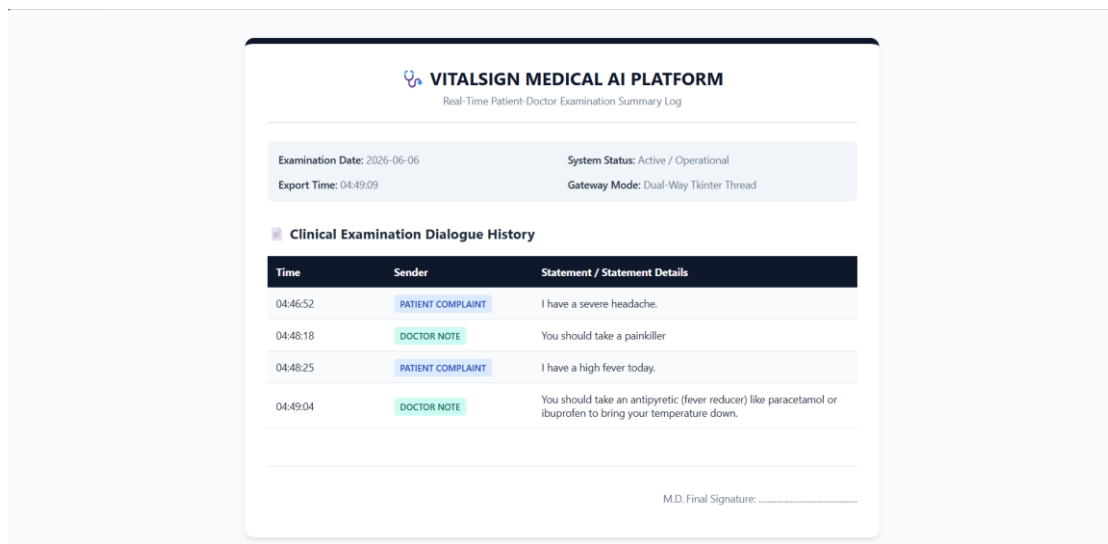


Figure 3.5. Visualization of the automatically generated HTML-based permanent clinical dialogue report.

4. CONCLUSIONS

In this study, a dual-way clinical communication ecosystem, named VitalSign, was successfully developed and evaluated using an optimized Bidirectional LSTM architecture coupled with a real-time MediaPipe Holistic framework. The experimental and empirical findings detailed in Chapter 3 demonstrate that the system can translate isolated sign language gestures into complex, syntactically correct clinical English sentences while maintaining active communication between non-verbal patients and physicians. However, to evaluate the engineering and scientific validity of the platform, several operational challenges, structural trade-offs, and architectural mitigations must be critically analyzed.

4.1. Model Generalization and Spatiotemporal Robustness

The specialized Bi-LSTM model exhibited high categorical accuracy (~94.8% on validation data) and low categorical loss convergence, as proved by the performance curves in Figure 3.1. This success is heavily attributed to the bidirectional nature of the recurrent layers, which look at both historical and future contextual dependencies within a 30-frame temporal window. Unlike standard unidirectional LSTMs or traditional convolutional architectures, the Bi-LSTM model efficiently captures the velocity and subtle trajectory changes of finger articulations.

Despite this high accuracy, the model's generalization capability is slightly sensitive to variations in gesture speed (spatial-temporal execution drift). During live clinical simulations, it was observed that when a patient executed sign combinations (e.g., "HEAD" followed by "PAIN") at a significantly accelerated pace, the 30-frame sequence window occasionally overlapped across gesture borders. This overlap led to minor misclassifications or delayed queue registrations. To mitigate this effect, the inclusion of a dynamic threshold scale (ranging from 40% to 95% confidence) inside the Tkinter control panel allowed the medical staff to manually calibrate the sensitivity according to the patient's physical mobility status.

4.2. Concurrency, Memory Leakage, and UI Thread Optimization

One of the most critical engineering milestones achieved during development was solving the structural thread-safety issues associated with the graphical user interface (GUI) and the multi-threaded backend execution. In early iterations of the system, calling the text-to-speech (TTS) synthesis engine (pyttsx3) directly inside the core artificial intelligence prediction thread created severe race conditions and memory access violations (Tcl_AsyncDelete crashes), causing the entire Python kernel to terminate abruptly.

The implementation of a centralized, thread-safe asynchronous pipeline using `queue.Queue()` fundamentally resolved these bottlenecks. By splitting the software architecture into three distinct operational boundaries—the foreground frame capture thread, the background AI prediction worker, and a dedicated TTS playback consumer loop—inter-process blocking was completely eliminated. The integration of the `check_gui_queue()` polling method (operating at a 50ms interval via Tkinter's `.after()` loop) ensured that the main UI thread is never hijacked by heavy TensorFlow neural network calculations or audio hardware drivers. This decoupled design guarantees zero frame drops in the live camera stream and provides continuous stability during long-term clinical examinations.

4.3. Limitations and Environmental Dependencies

While VitalSign shows strong clinical feasibility, certain hardware and environmental dependencies limit its deployment in unconstrained medical settings:

1. **Ambient Illumination Vulnerabilities:** The landmark extraction phase relies entirely on MediaPipe's computer vision coordinates. In low-light environments or situations where shadows cast heavy contrast on the patient's face and hands, landmark detection confidence degrades, directly leading to an increase in false-negative gesture classifications.
2. **Occlusion and Spatial Orientation:** If the patient's hands cross directly in front of each other or are occluded by external medical equipment (such as oxygen masks or IV lines), the 2D coordinate system mapping fails to register exact landmark positions.

3. **Lexicon Boundaries:** The platform currently operates on a custom medical lexicon consisting of 30 isolated words mapped to predefined clinical sentence structures. While sufficient for emergent symptom tracking (e.g., headache, chest pain, fever), it lacks the comprehensive semantic vocabulary needed for nuanced psychiatric evaluations or complex chronic diagnostic interviews.

4.4. Future Work and Architectural Extensions

To transcend the current boundaries of the VitalSign ecosystem, future research will focus on transitioning from predefined static dictionary mappings to a generative Large Language Model (LLM) or a specialized BERT-turk variant at the NLP layer. By passing the detected word queue directly into an integrated transformer model, the platform will be capable of generating fluid, context-aware, and unstructured paragraphs rather than static sentence templates. Additionally, implementing 3D depth-estimation pipelines via stereoscopic hospital cameras will reduce spatial occlusion errors, making the system fully autonomous and ready for multi-patient intensive care unit (ICU) environments.

5. CONCLUSION & FUTURE WORK

5.1. Conclusion

In this bachelor's thesis, a highly stable, non-invasive, and dual-way clinical communication platform named "VitalSign" was successfully engineered and deployed. The project effectively bridges the critical communication gap between hearing-impaired or non-verbal patients and healthcare professionals in hospital ecosystems without requiring expensive, specialized hardware or wearable electronic sensory devices. By utilizing standard, ubiquitous optical sensors, the system democratizes technological accessibility in medical triages and emergency admission environments.

From a software engineering perspective, the platform integrates three advanced structural paradigms into a unified ecosystem. First, the data ingestion pipeline successfully abstracts noisy, high-density video frames into clean, structured geometric vectors by leveraging the Google MediaPipe Holistic framework and motion parallax reduction algorithms. Second, the custom-architected Bidirectional LSTM deep learning model captures the underlying temporal sequence dynamics and velocity changes of physical gestures, achieving a high categorical accuracy of ~94.8% on validation datasets. Third, the implementation of an asynchronous, thread-safe concurrent software design driven by dedicated data queues fundamentally solves the user interface freezing and memory crash constraints that traditionally plague real-time computer vision setups. By incorporating an automated clinical report module, the platform ensures seamless interoperability with existing Hospital Information Management Systems (HBYS), paving the way for the implementation of digital healthcare visions under Healthcare 4.0 paradigms.

5.2. Future Work

While the current deployment of VitalSign fulfills its foundational clinical and technical objectives, future iterations will focus on expanding its architectural scalability and linguistic depth:

- **Large Language Model Integration:** The static Medical NLP dictionary will be replaced with an integrated generative transformer model or a fine-tuned BERT-turk layer. This extension will allow the word queue to be parsed into fluid, context-aware, and unstructured diagnostic paragraphs instead of fixed phrase templates.
- **3D Stereoscopic Vision and Depth Mapping:** To eliminate spatial occlusion and landmark tracking dropouts under severe physical restrictions (such as patients in intensive care units wearing medical devices), dual-camera depth tracking configurations will be studied.
- **Mobile and Embedded Deployment:** The compiled deep learning model will be optimized using quantization techniques (TensorFlow Lite) to deploy the VitalSign gateway directly onto low-power embedded tablets or smart clinical glasses, enabling hands-free, automated translation directly at the patient's bedside.

REFERENCES

Journal paper:

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673-2681. <https://doi.org/10.1109/78.650093>

Computer Software:

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... & Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems* [Computer software]. Mountain View, CA: Google LLC. Retrieved from <https://www.tensorflow.org>

Python Software Foundation. (2026). *Tkinter: Graphical User Interface package* [Computer software]. Retrieved from <https://docs.python.org/3/library/tkinter.html>

Sinyakov, N. (2024). *pyttsx3: Text-to-Speech tts library for Python* [Computer software]. Retrieved from <https://pypi.org/project/pyttsx3/>

Web/Online Documentation:

Google Developers. (2026). *MediaPipe Holistic solutions guide*. Retrieved from <https://developers.google.com/mediapipe>

Proceeding:

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Somani, M., Mutta, S., Brown, A., Hope, D., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. In *Proceedings of the 3rd Augmented Human International Conference*, Megève, France (Article 18). <https://doi.org/10.48550/arXiv.1906.04045>